

Binôme :

- Victor NOUANE
- Didier TERRIER

Projet NFE204

Conservatoire national des Arts et Métiers

SUJET : SOLR avec CARROT2

Février 2015



Table des matières

1. Présentation générale du projet :	3
2. Présentation des documents utilisés pour le projet :	4
a) Description des documents	4
b) Recueil et formatage des documents	4
3. Utilisation du moteur de recherche « Solr » :	6
a) Installation de Solr	6
b) Configuration du schema.xml	7
c) Stockage des documents dans Solr	8
4. Utilisation du module de classification Carrot ² :	9
a) Présentation de Carrot ²	9
b) Carrot ² dans le projet.....	11
c) Tests avec Carrot ²	12
Test 1 : requête avec le mot “malade” avec un résultat de 100 documents: .	13
Tests 2 à 5 : requête avec le même mot “malade” avec des résultats entre 200 et 290 documents:	14
Tests 6 et 7 : requête avec le même mot “malade” avec des résultats 300 et 500 documents:	16
Test 8 : requête avec les mots “un malade” avec 100 résultats:	18
Test 9 : requête avec les mots “animal malade” avec 100 résultats:.....	19
Test 10 : requête avec les mots “enfant malade” avec 100 résultats:	20
d) Constats après les tests avec Carrot ²	22
e) Intégration de Carrot ² dans Solr	23
5) Conclusion :	26

1. Présentation générale du projet :

Le domaine que nous avons choisi d'étudier pour le projet NFE204 est la recherche documentaire et en particulier les techniques de similarité.

C'est une partie du cours NFE204 qui nous a beaucoup intéressés et les champs d'application sont très vastes.

Nous commencerons par présenter les documents que nous avons sélectionnés pour le projet. Puis, nous décrirons les étapes de transformation de ces documents qui ont été nécessaires pour permettre leur utilisation dans le cadre de notre étude.

Nous traiterons ensuite du choix de Solr comme base de ces documents et de son intégration avec le module Carrot² qui est un moteur de classification des résultats de recherche open source. Nous ne connaissions absolument pas ce module, nous allons donc en profiter pour l'étudier, le présenter et l'appliquer à la base documentaire Solr initialement constituée.

Nous procéderons à différents tests pour mettre en évidence les techniques de classification disponibles dans le module Carrot².

Enfin, nous terminerons par un bilan des différents apports obtenus grâce à la réalisation de ce projet.

2. Présentation des documents utilisés pour le projet :

a) Description des documents

Nous cherchons un site qui permet d'extraire des documents avec leur texte qui sont dans le domaine public. Nous avons repéré le site du projet « Gutenberg ».

<http://www.gutenberg.org/>

Il s'agit d'une bibliothèque de livres en versions électroniques et faisant partie du domaine public.

Ce projet propose plus de 40 000 livres en plusieurs langues et en libre accès.

b) Recueil et formatage des documents

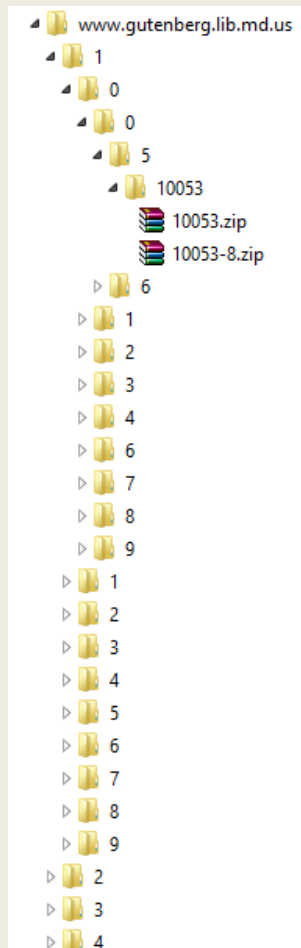
Pour le projet NFE204, nous nous limitons aux livres français et au format txt.

Pour télécharger les livres, nous utilisons l'utilitaire « wget » sous Windows, avec la commande suivante :

```
wget -H -w 2 -m "http://www.gutenberg.org/robot/harvest?filetypes[]=txt&langs[]=fr" -  
-referer="http://www.google.com" --user-agent="Mozilla/5.0 (Windows; U; Windows  
NT 5.1; en-US; rv:1.8.1.6) Gecko/20070725 Firefox/2.0.0.6" --header="Accept:  
text/xml,application/xml,application/xhtml+xml,text/html;q=0.9,text/plain;q=0.8,image  
/png,*/*;q=0.5" --header="Accept-Language: en-us,en;q=0.5" --header="Accept-  
Encoding: gzip,deflate" --header="Accept-Charset: ISO-8859-1,utf-8;q=0.7,*;q=0.7" --  
header="Keep-Alive: 300"
```

Dans le requête ci-dessus nous avons spécifié le type de fichier désiré (txt) et la langue des textes (fr).

Finalement, nous obtenons un dossier `www.gutenberg.lib.md.us/` avec de nombreux fichiers dans une arborescence du type `\1\0\0\5\10053\10053.zip`.



Nous récupérons 3387 fichiers, soit environ 540Mo de données zippées.

Puis nous utilisons un programme Java pour transformer les fichiers texte en fichier XML de la forme suivante :

```
<?xml version="1.0" encoding="UTF-8" standalone="no"?>
<add>
  <doc>
    <field name="id">{id}</field>
    <field name="title">{titre}</field>
    <field name="author">{auteur}</field>
    <field name="content">{texte}</field>
```

</doc>
</add>

Le programme Java supprime également les textes en doublon. Le texte contenu dans les archives 10053.zip et 10053-8.zip sont similaires. Ils ne seront alors indexés qu'une seule fois.

3. Utilisation du moteur de recherche « Solr » :

Nous avons choisi le moteur Solr comme support de base pour les documents du projet pour 2 raisons principales :

- c'est un moteur qui fonctionne sous windows,
- et il est supporté par le module de classification Carrot².

a) Installation de Solr

Nos machines tournant sous Windows, nous nous sommes tournés vers la version Bitnami Solr 4.10.3 .

Voici le lien permettant de télécharger la version 4.10.3 de Solr :

<https://bitnami.com/redirect/to/47634/bitnami-solr-4.10.3-0-windows-installer.exe>

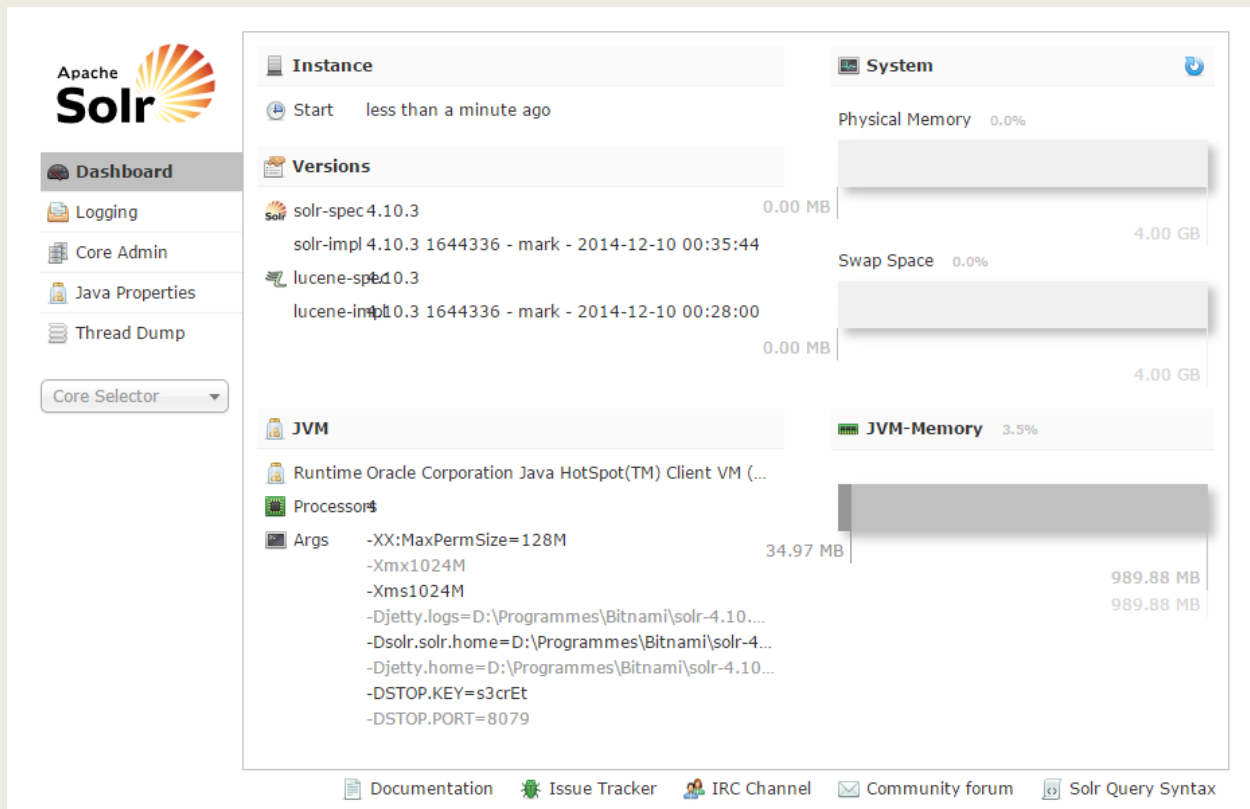
Il s'agit d'une installation classique d'un programme Windows.

A la fin de l'installation, deux services sont créés :

solrApache : permet de démarrer le serveur web Apache de Solr

solrJetty : permet de démarrer le serveur Solr

Une fois, ces deux serveurs démarrés, nous utilisons le serveur web classique de Solr à l'url <http://localhost:8983/solr> .



b) Configuration du schema.xml

Nous créons un index que nous appelons nfe204.

Dans le fichier schema.xml, nous ajoutons les champs dont nous avons besoins et utilisons le type text_fr.

```
<field name="id" type="string" indexed="true" stored="true" required="true"
multiValued="false" />
<field name="title" type="text_fr" indexed="true" stored="true"
```

```

multiValued="true"/>
  <field name="author" type="text_fr" indexed="true" stored="true"/>
  <field name="content" type="text_fr" indexed="false" stored="true"
multiValued="true"/>
  <field name="text" type="text_fr" indexed="true" stored="false"
multiValued="true"/>

  <fieldType name="text_fr" class="solr.TextField" positionIncrementGap="100">
    <analyzer>
      <tokenizer class="solr.StandardTokenizerFactory"/>
      <filter class="solr.LowerCaseFilterFactory"/>
      <filter class="solr.ElisionFilterFactory" ignoreCase="true"
articles="lang/contractions_fr.txt"/>
      <filter class="solr.StopFilterFactory" ignoreCase="true"
words="lang/stopwords_fr.txt" format="snowball" />
      <filter class="solr.SnowballPorterFilterFactory"
language="French"/>
      <filter class="solr.ASCIIFoldingFilterFactory"/>
    </analyzer>
  </fieldType>

```

c) Stockage des documents dans Solr

Nous indexons ensuite l'ensemble des documents xml à l'aide du programme post.jar fourni dans exampledocs avec la commande suivante :

```
java -jar post.jar *.xml .
```




- Dashboard
- Logging
- Core Admin
- Java Properties
- Thread Dump
- nfe204
- Overview
- Analysis
- Dataimport
- Documents
- Files
- Ping
- Plugins / Stats
- Query
- Replication
- Schema Browser

Statistics

Last 2 days ago
Modified:
Num Docs: 2487
Max Doc: 2487
Heap 500696
Memory
Usage:
Deleted 0
Docs:
Version: 103
Segment 12
Count:
Optimized: 
Current: 

Replication (Master)

	Version	Gen	Size
Master (Searching)	1424461937438	29	863.9 MB
Master (Replicable)	-	-	-

Admin Extra

Instance

CWD: D:\Programmes\Bitnami\solr-4.10.3-0\java\bin
Instance: D:\Programmes\Bitnami\solr-4.10.3-0\apache-solr\solr\nfe204
Data: D:\Programmes\Bitnami\solr-4.10.3-0\apache-solr\solr\nfe204\data
Index: D:\Programmes\Bitnami\solr-4.10.3-0\apache-solr\solr\nfe204\data\index
Impl: org.apache.solr.core.NRTCachingDirectoryFactory

Healthcheck

Ping request handler is not configured with a healthcheck file.

Documentation

Issue Tracker

IRC Channel

Community forum

Solr Query Syntax

4. Utilisation du module de classification Carrot² :

a) Présentation de Carrot²

Carrot² est une application java développée à partir de 2001 par Lingo Dawid Weiss et Stanislaw Osinski à l'Université de Technologie de Poznan, en Pologne. Carrot² est composé d'un certain nombre de modules qui couvrent chacun des processus de text mining.

Carrot² est un moteur de classification des résultats de recherche open source. Il peut automatiquement regrouper des documents d'une collection à partir d'une recherche dans des catégories thématiques.

Carrot² a été conçu avec le développement de l'Internet et le problème classique de récupération de renseignements qui est apparu: étant donné un ensemble de documents et une requête, déterminer le sous-ensemble de documents pertinents à la requête.

Carrot² est un ensemble de bibliothèques et d'applications permettant de créer un moteur de recherche des résultats de classification (ou clustering). Un tel moteur organise les résultats de recherche par catégorie, entièrement automatique et sans connaissance externe comme les taxonomies ou contenu pré-classifié.

Carrot² possède également deux algorithmes de classification conçus spécifiquement pour les résultats de recherche qui se nomment Lingo (prénom de l'un des 2 créateurs Lingo Dawid Weiss), et STC (Suffix Tree Clustering). Nous reviendrons dans le détail sur des deux algorithmes car ils constituent les éléments importants de notre projet NFE204.

Carrot² sait traiter 20 langues (la plupart des langues européennes ainsi que l'arabe, le chinois, japonais et coréen).

Carrot² propose aussi des composants pour l'extraction de données des moteurs de recherche qui fournissent les API nécessaires (par exemple Microsoft Bing), ainsi que d'autres sources de documents comme Lucene, Solr Apache ou ElasticSearch.

Carrot² n'est pas un moteur de recherche, il ne possède pas de système de moteur et pas de système d'indexe. C'est pourquoi, il peut s'appuyer sur des moteurs de type Solr.

Comme Carrot² n'a pas de moteur de recherche natif, il n'y a pas de syntaxe commune de requête dans Carrot². La syntaxe dépend du moteur de recherche externe qu'on utilise avec Carrot² (via un paramétrage), par exemple, Google, Bing, Solr, Lucene ou tout autre. Carrot² passe la requête sans aucune modification au moteur de recherche puis Carrot² catégorise les résultats de la requête fournis par le moteur de recherche externe. C'est pour cette raison que

toute syntaxe prise en charge par le moteur de recherche est automatiquement prise en charge par Carrot². Il ne fait aucun filtrage ni transformation à ce niveau.

Comme indiqué précédemment, Carrot² est livré avec un ensemble d'outils et d'API qui sont :

- Carrot² document Clustering Workbench : qui est une application graphique complète et autonome à installer facilement sur le poste utilisateur pour effectuer de la classification avec les algorithmes de Carrot²,
- une API Java et C# de Carrot2 pour appeler les algorithmes de classification de Carrot² à partir d'un code Java, C # ou .NET,
- un «Carrot² document Clustering server » qui expose les algorithmes de classification de Carrot² comme un service REST,
- une interface de commande en ligne pour invoquer de cette manière les algorithmes de classification de Carrot²,
- et enfin il existe aussi une application Web Carrot² pour les utilisateurs finaux.

Il existe une version payante (appelée Lingo3G) qui propose une variante de l'algorithme Lingo bien plus puissante et précise. Elle s'appuie sur la technique de « clustering hiérarchique », sur un algorithme plus complexe de définition des noms des catégories et sur la gestion des synonymes. Enfin, cette version payante est bien plus performante pour les collections de grande taille. Bien sûr, pour les besoins du projet NFE204, la version gratuite de Carrot² est amplement suffisante.

b) Carrot² dans le projet

Pour le projet NFE204, nous avons choisi d'utiliser l'application gratuite « Carrot² document Clustering Workbench » afin de pouvoir l'installer sur l'ordinateur contenant déjà :

- la collection de documents Gutenberg au format texte et xml (après transformation via un programme java que nous avons développé),
- et le moteur de recherche et base Solr.

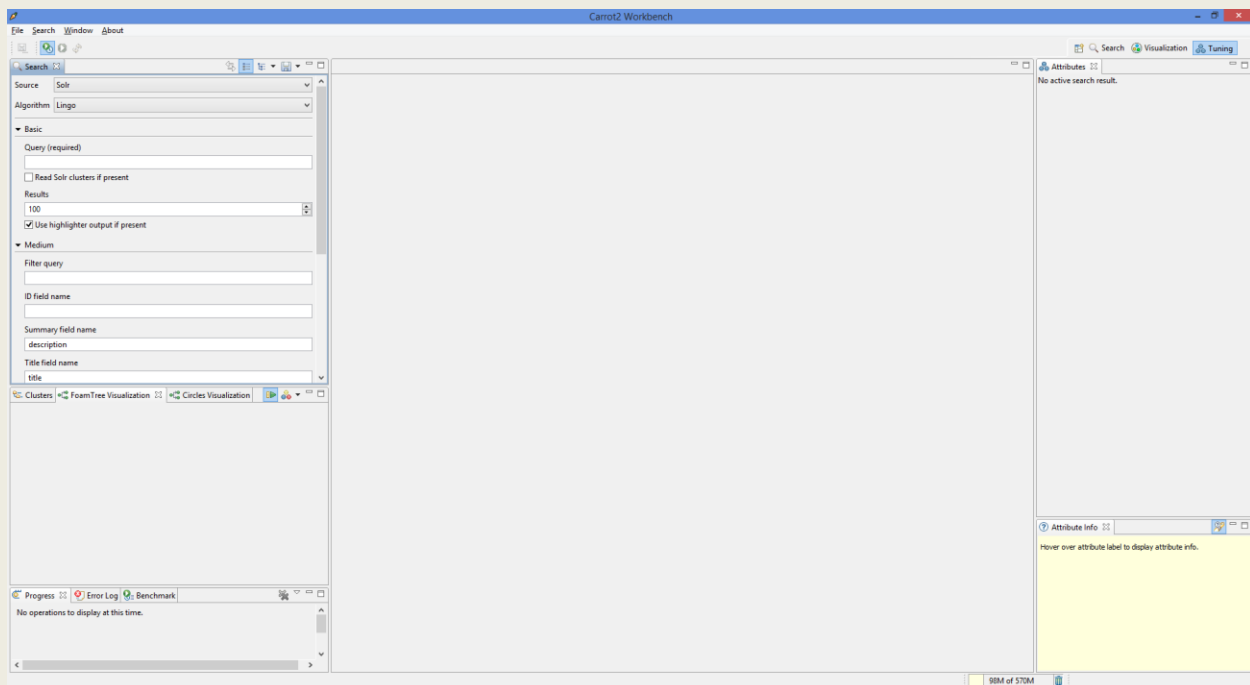
L'ordinateur est un PC Microsoft sous Windows 8 avec RAM de 8Go.

Nous avons procédé de la façon suivante pour l'installation de l'application « Carrot² document Clustering Workbench » :

- site <http://project.carrot2.org/download.html>
- téléchargement de la version la plus récente « 3.9.4 » en windows 64 bits
- exécution de l'assistant d'installation avec les options par défaut

Nous n'avons pas rencontré de difficulté.

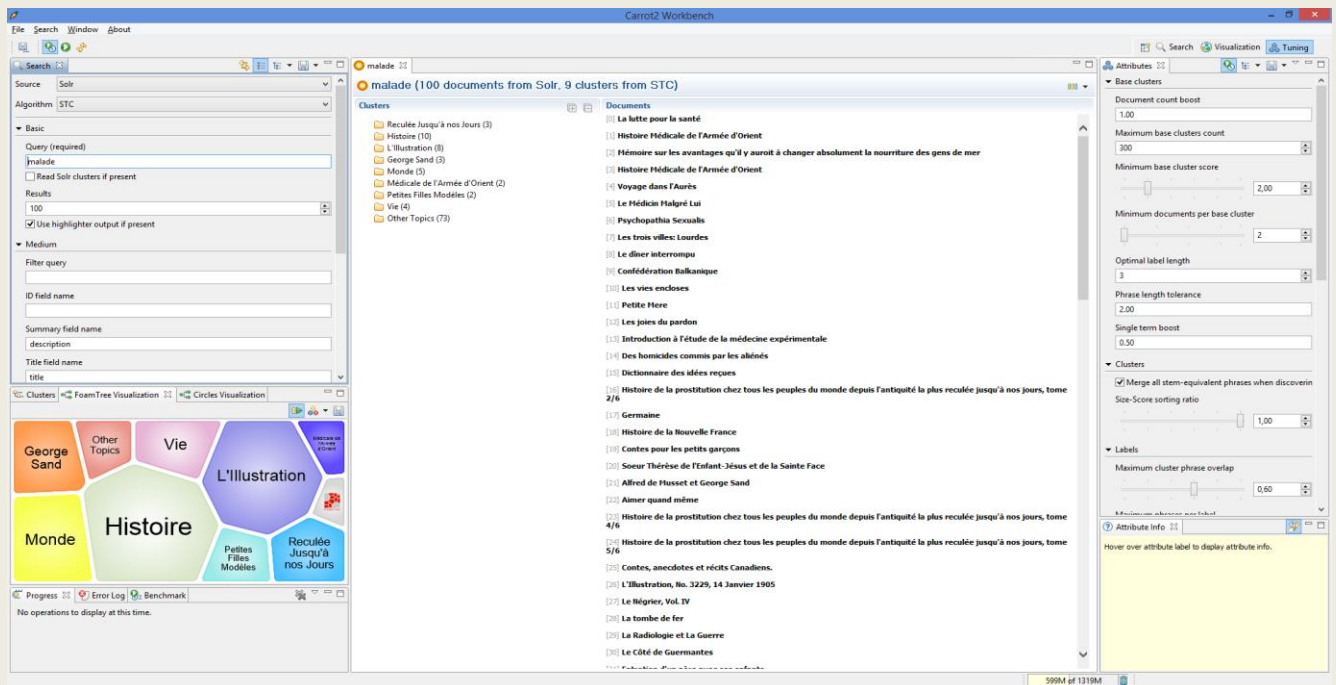
Puis, nous avons lancé l'application.



Puis, via l'interface, nous avons sélectionné comme source « Solr » et préciser la base contenant notre collection de documents.

c) Tests avec Carrot²

Ne connaissant absolument pas Carrot², mais comme le cours de NFE204 sur les thèmes « base documentaire » et « recherche avec classement » nous avait fortement



Résultats pour 100 documents :

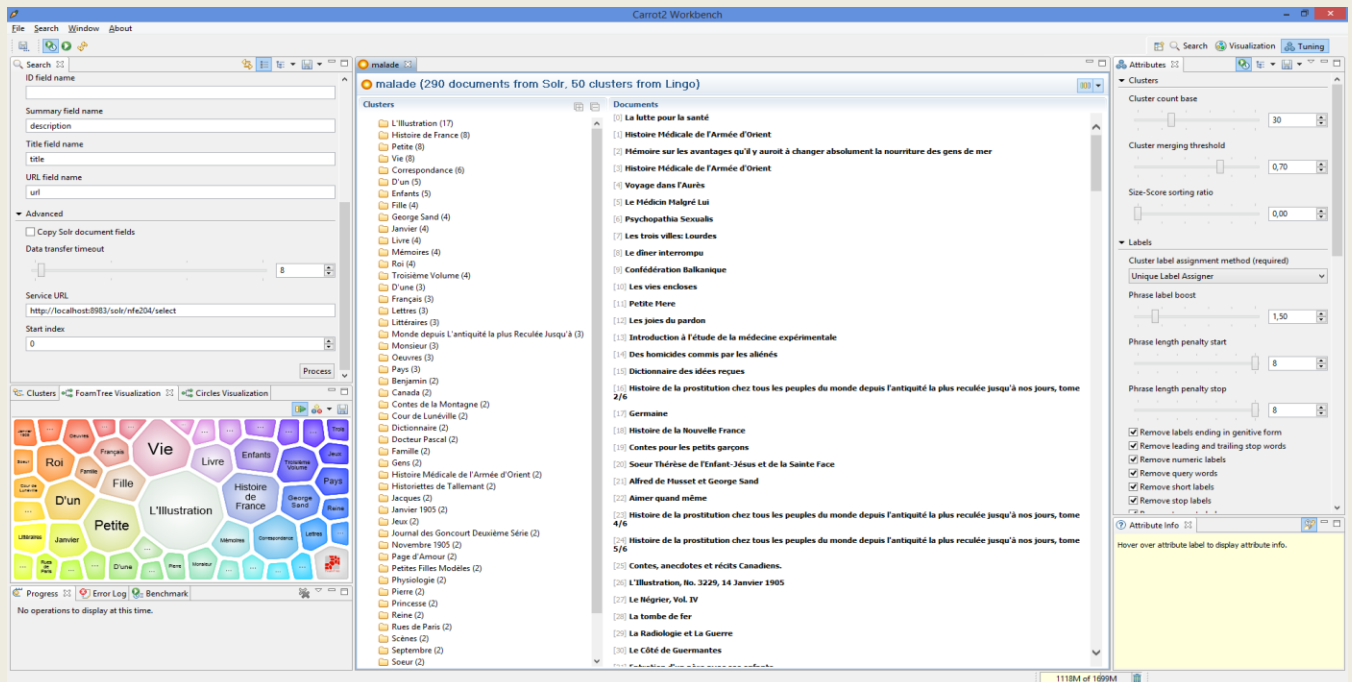
- les 2 algorithmes sont rapides,
- avec Lingo, on obtient 22 clusters (ou catégories),
- avec STC, 9 clusters,
- du coup, c'est bien plus précis avec Lingo qu'avec STC,
- même thème dominant « Histoire » avec 10 documents pour les 2 algorithmes,
- 5 clusters en commun : Histoire, L'illustration, Georges Sand, Monde, Vie.
- pour STC, le thème dominant est « Enfants » avec 7 documents, puis « Histoire » avec 6 documents, puis « Contes » et « Petite » avec 4 documents.
- On remarque aussi le cluster « Other Topics » : qui est considéré comme la catégorie « Autre » pour les documents ayant le mot « malade » mais n'ayant pu être bien classés dans les premières catégories
 - Pour Lingo : 58 documents, soit 58% non classés
 - Et pour STC : 73 documents, soit 73% non classés

Tests 2 à 5 : requête avec le même mot “malade” avec des résultats entre 200 et 290 documents:

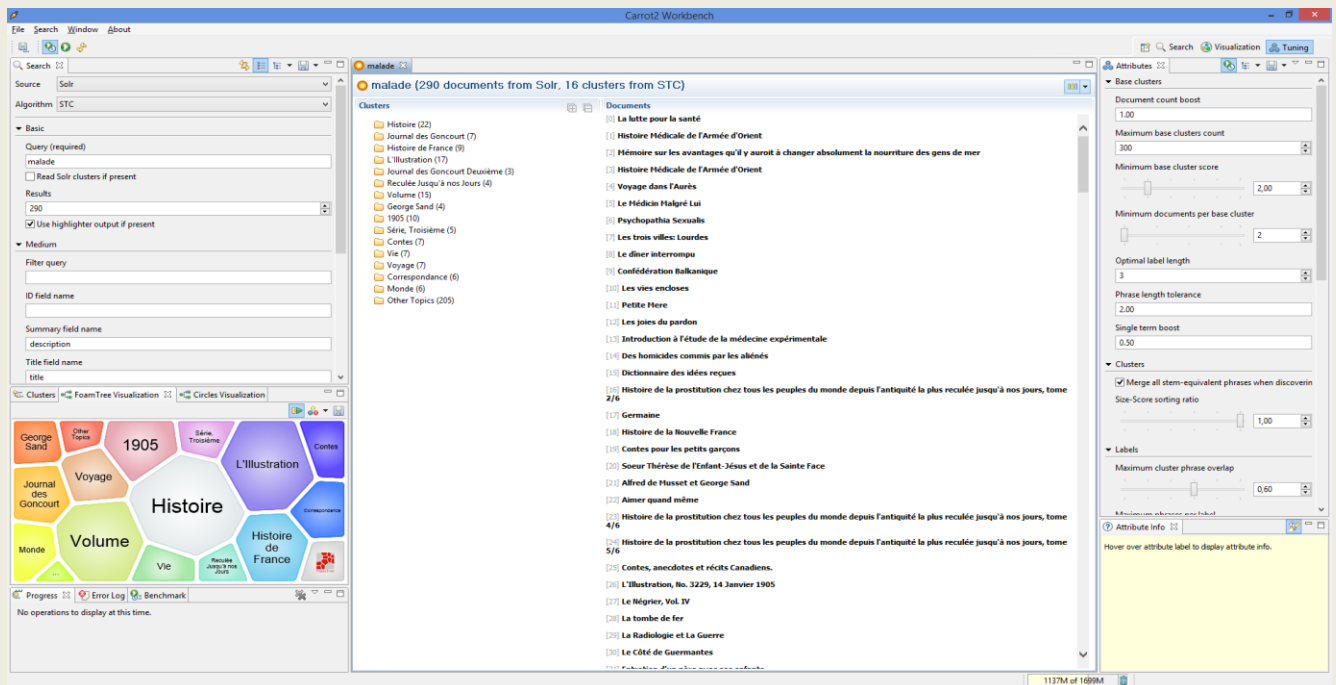
Résultats :

- Résultats avec 200 documents : Lingo: 43 clusters, STC: 16 clusters
- Résultats avec 250 documents : Lingo: 48 clusters, STC: 16 clusters
- Résultats avec 270 documents : Lingo: 50 clusters, STC: 16 clusters
- Résultats avec 280 documents : Lingo: 51 clusters, STC: 16 clusters
- Résultats avec 290 documents : Lingo: 52 clusters, STC: 16 clusters
- Avec STC, il y a une rapide stagnation à 16 clusters alors qu'avec Lingo, la classification continue à s'affiner.

Copie d'écran : résultats avec 290 documents, Lingo: 52 clusters,



Copie d'écran : résultats avec 290 documents, STC: 16 clusters,



On constate que :

- pour STC, le thème dominant « Histoire » demeure avec 22 documents au lieu de 10, puis encore en deuxième place « L'illustration » avec 17 documents.
- Avec Lingo, c'est « L'illustration » avec 17 documents en tête à la place de « Histoire ». « Petites », « Vie », « Enfants » avec 8 et 6 documents demeurent en tête alors que « Contes » a disparu.
- Concernant le cluster « Other Topics » :
 - Pour Lingo : 153 documents soit 53% (au lieu de 58% au début)
 - Pour STC : 205 documents soit 71% (au lieu de 73% au début)

Tests 6 et 7 : requête avec le même mot “malade” avec des résultats 300 et 500 documents:

Avides de connaître les limites de ces deux algorithmes, nous poursuivons nos tests avec 300 résultats. Mais, nous obtenons un message d'erreur de mémoire. Nous approfondissons nos investigations et nous nous rendons compte que le message d'erreur provient en fait de Solr. Ceci est confirmé en effectuant la même requête directement via l'interface de Solr. Grâce à des recherches sur Internet, on change des paramètres de configuration de Solr en rajoutant ces 2 options :

JvmOptions=Xms1024M

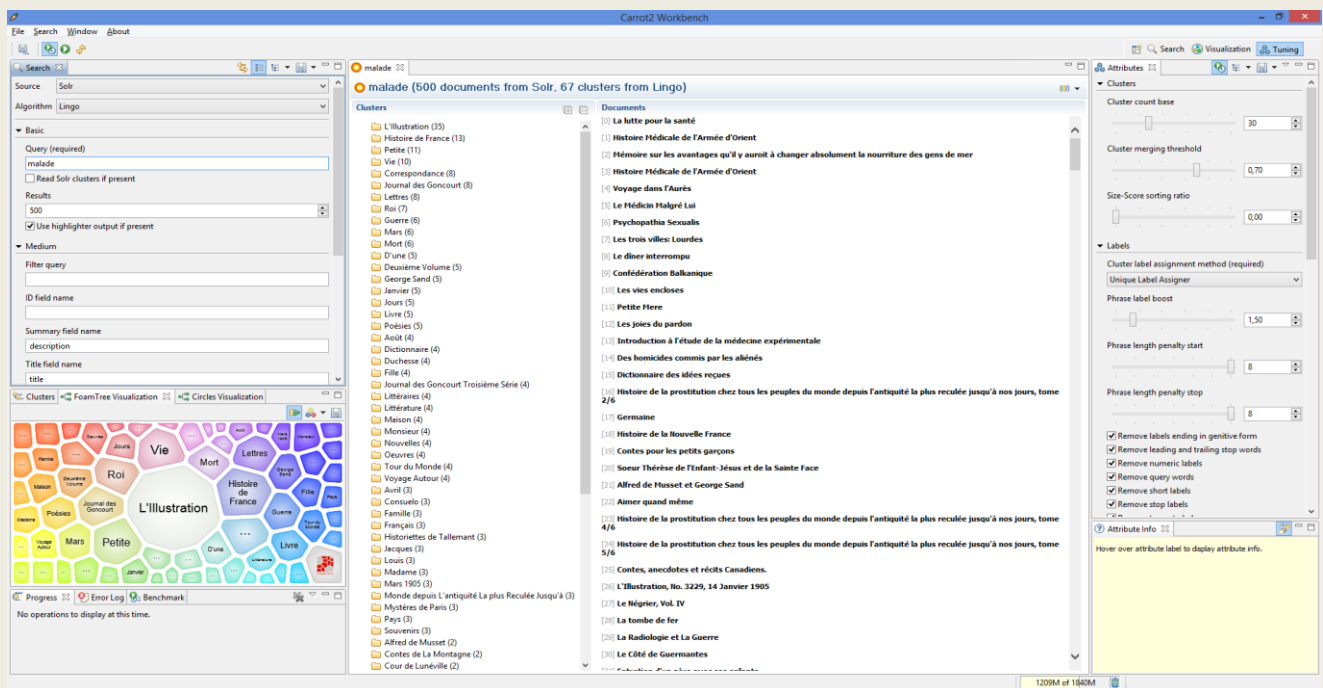
JvmOptions=Xmx1024M

On retente avec Solr, Eureka ! la requête passe, idem avec Carrot².

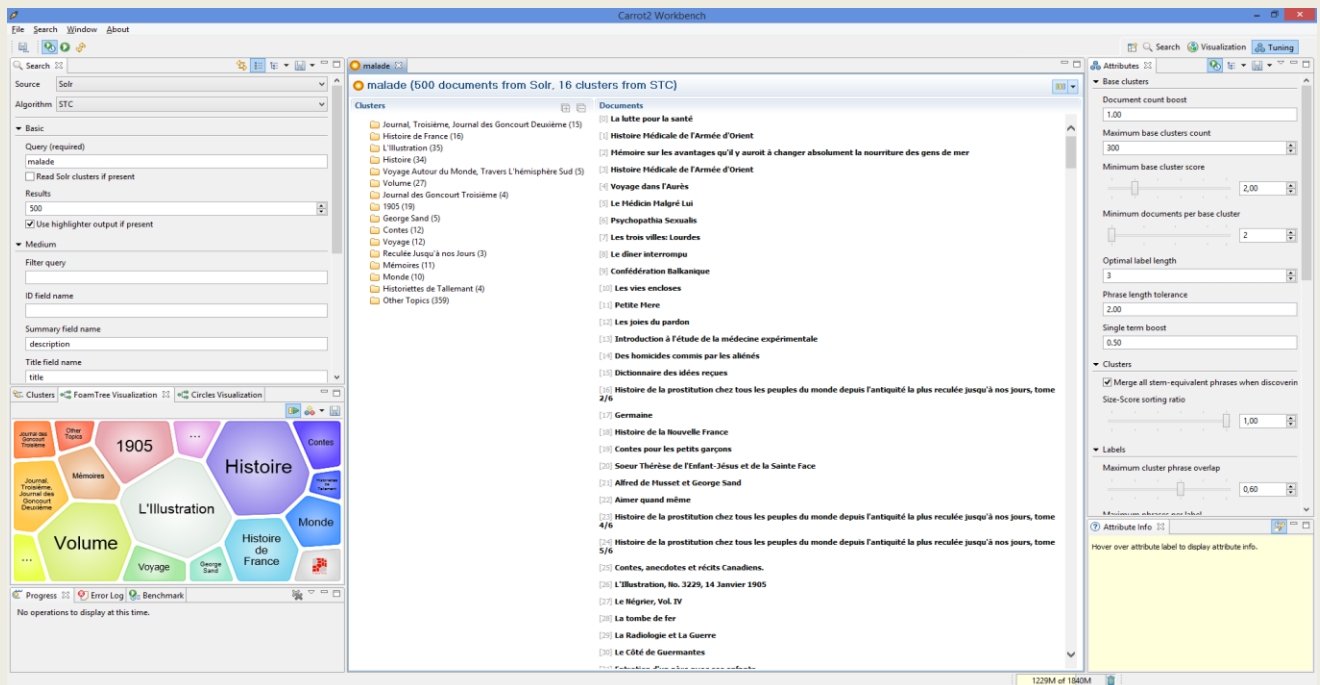
Résultats :

- Résultats avec 500 documents : Lingo: 64 clusters, STC: toujours 16 clusters
- La stagnation de STC persiste, alors que Lingo poursuit sa classification (+12 clusters)
- Au-delà de 500 documents, nous avons de nouveau le message d'erreur de mémoire mais nous atteignons les limites de notre PC. Nous ne pouvons aller au-delà.

Copie d'écran : résultats avec 500 documents, Lingo: 64 clusters,



Copie d'écran : résultats avec 500 documents, STC: 16 clusters



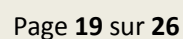
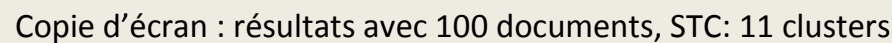
On constate que :

- pour STC, le thème dominant a encore changé, cette fois-ci c'est « L'Illustration » avec 35 documents est devant de très peu par rapport à « Histoire » avec 34 documents suivi de « Histoire de France » avec 16 documents. Quant à « Petites », « Vie », « Enfants », ils ont totalement disparu par rapport à « Volume » 27 documents et « 1905 » avec 19 documents.
- Avec Lingo, « L'Illustration » assure nettement sa première place avec 35 documents suivi de « Histoire de France » avec 13 documents. « Petites » et « Vie » avec 11 et 10 documents demeurent en tête.
- Concernant le cluster « Other Topics » :
 - Pour Lingo : 282 documents soit 56% (au lieu de 58% au début)
 - Pour STC : 359 documents soit 72% (au lieu de 70% au début)

Test 8 : requête avec les mots “un malade” avec 100 résultats:

Pour les 2 algorithmes, nous obtenons exactement les mêmes résultats qu'avec le test 1 « requête malade » tant dans les noms des clusters que dans les documents. C'est logique puisque Carrot² applique aussi une normalisation notamment l'étape des « stop words ».

Copie d'écran : résultats avec 100 documents, Lingo: 31 clusters,



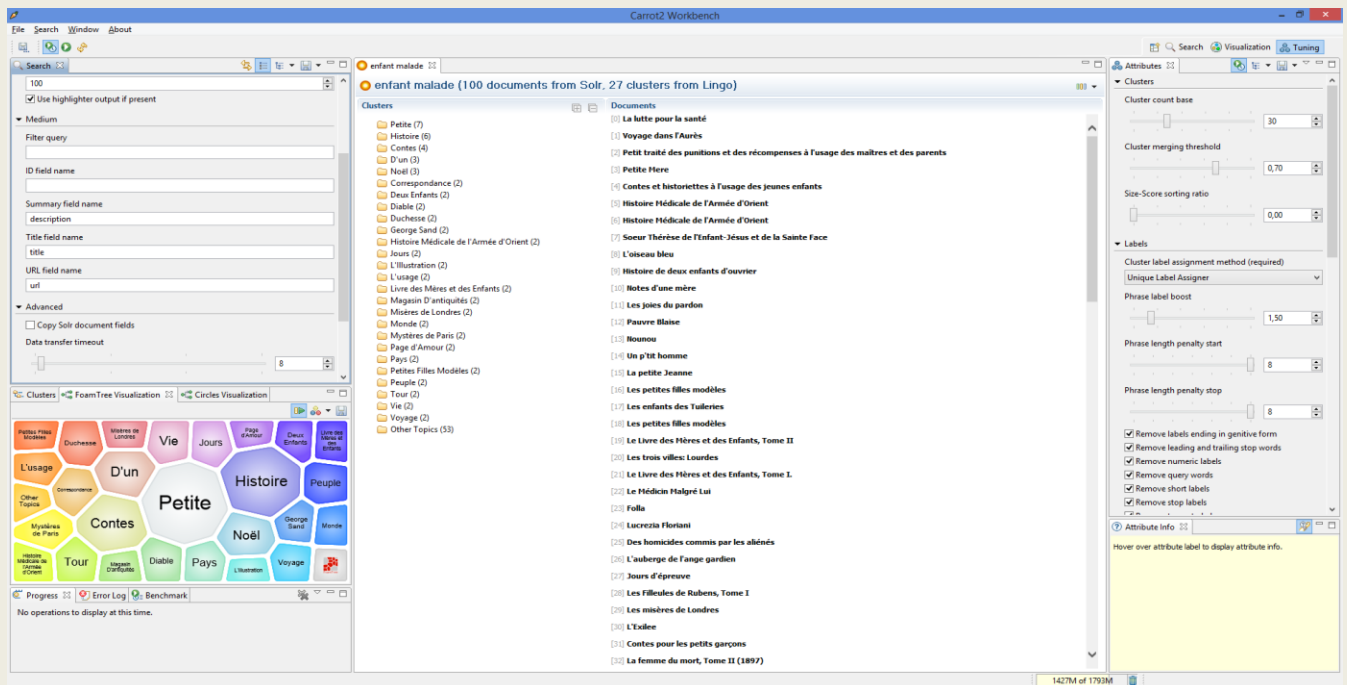
En comparant cette requête avec la requête initiale « malade », on constate que :

- au niveau des clusters :
 - avec Lingo, on obtient 31 cluster (22 avec « malade »)
 - avec STC, 11 clusters (7 avec « malade »)
- au niveau du cluster dominant :
 - avec la requête « animal malade » : « L'illustration » avec 12 documents pour Lingo et STC
 - avec la requête « malade » : « Histoire » avec 10 documents pour Lingo et STC
- Nouveaux thèmes avec la requête « animal malade »
 - Lingo : Voyage (10 documents), Monde (4 documents), Mer (4 documents), Noir (3 documents), Pays, Aventure, Tour du Monde : 2 documents
 - STC : Voyages (10 documents), Mers, Monde (4 documents)
- Au niveau du cluster « Other Topics » :
 - requête « animal malade » : Lingo: 40%, STC: 58%
 - requête « malade » : Lingo: 58%, STC: 73%

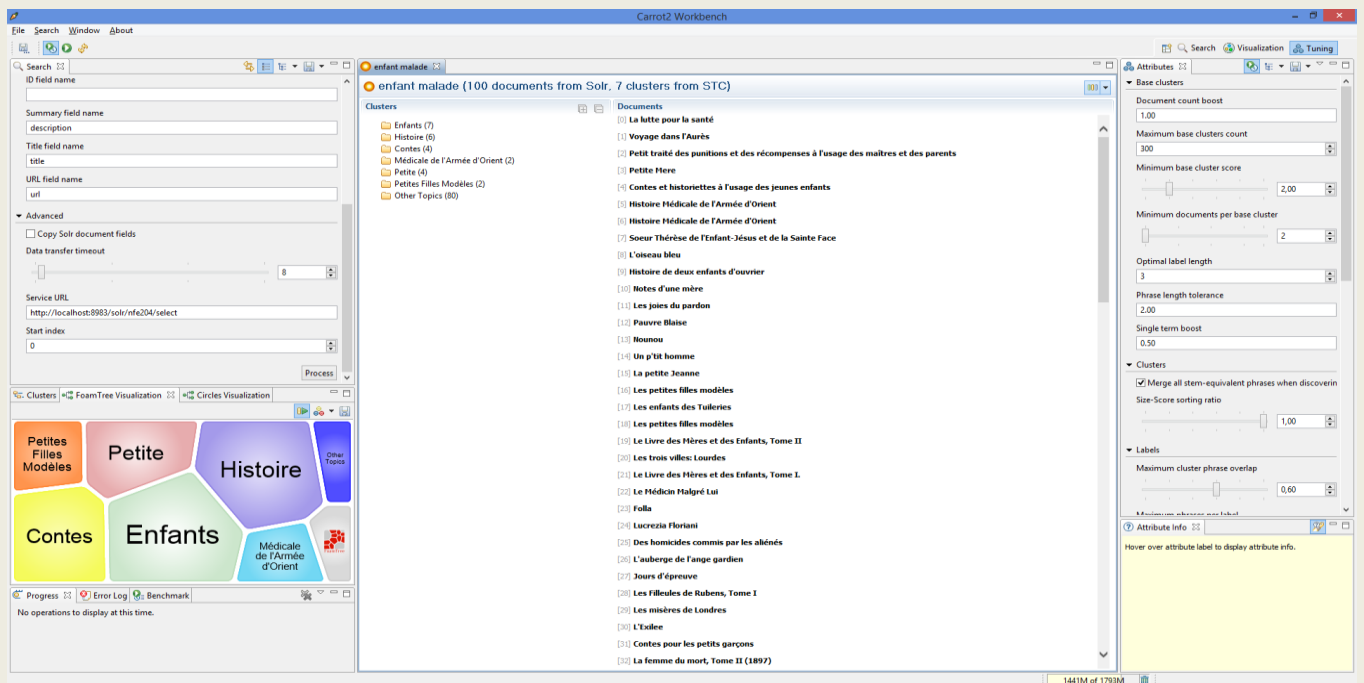
En conclusion, le mot supplémentaire « animal » a apporté un sens plus précis à « malade » et a ainsi fait apparaître d'autres catégories autour du voyage, l'aventure et la mer. Du coup, des thèmes comme « Vie » et « Enfants » plus orientés vers l'être humain ont disparu.

Test 10 : requête avec les mots "enfant malade" avec 100 résultats:

Copie d'écran : résultats avec 100 documents, Lingo: 27 clusters,



Copie d'écran : résultats avec 100 documents, STC: 7 clusters



En comparant cette requête avec la requête initiale « malade », on constate que :

- au niveau des clusters :

- avec Lingo, on obtient 27 cluster (22 avec « malade »)
- avec STC, 7 clusters (7 avec « malade »)
- au niveau du cluster dominant :
 - avec la requête « enfant malade » : « Petite » avec 6 documents suivi de « Histoire » avec 6 documents pour Lingo. Avec STC, en premier « Enfants » avec 7 documents, puis « Histoire » 6 documents
 - avec la requête « malade » : « Histoire » avec 10 documents pour Lingo et STC, puis « L'Illustration » avec 8 documents.
- Nouveaux thèmes avec la requête « enfant malade »
 - Il n'y en a pas beaucoup avec Lingo : on voit apparaître « Noel », « Peuple », « Correspondance », tout en retrouvant « Petite », « Histoire » et « Contes ».
 - Avec STC, aucun nouveau thème, en plus de « Enfants » et « Histoire », on retrouve « Contes » et « Petites ».
- Au niveau du cluster « Other Topics » :
 - requête « enfant malade » : Lingo: 53%, STC: 80%
 - requête « malade » : Lingo: 58%, STC: 73%

En conclusion, le mot supplémentaire « enfant » n'a pas apporté de grande précision à « malade » contrairement à « animal malade ». En particulier avec l'algorithme STC, aucun nouveau thème n'a été créé. On retrouve les catégories comme « Histoire », « Contes » et « Enfants ». Une précision avec « Noel ». D'ailleurs, si on examine la liste des documents, on retrouve un grand nombre en commun entre « enfant mlade » et « malade ». Cela signifie qu'au moins les 100 documents retournés comme résultat avec la requête « malade » portent sur l'être humain voire sur l'enfant.

d) Constats après les tests avec Carrot²

Grâce à ces différents tests, nous avons remarqué les points suivants sur Carrot² :

- Carrot² effectue les regroupements en mémoire. Il a donc besoin de beaucoup de mémoire (d'où nos messages d'erreur). Pour cette raison, Carrot² ne peut traiter plus d'un millier de documents avec quelques paragraphes chacun.
- Il faut fournir un minimum de documents (au moins 100) pour que les algorithmes de clustering de Carrot² agissent efficacement. De plus, Carrot² semble mieux fonctionner avec un ensemble de documents similaires avec des extraits proches des requêtes au lieu de documents complets ayant quelques parties ayant un lien avec les requêtes.
- On s'est rendu que la qualité des documents est importante. Lors de notre phase de transformation en xml, nous avons parfois tronqué des morceaux de phrases, ce qui a influencé le clustering.
- L'algorithme Lingo semble plus précis que l'algorithme STC. Il crée plus de catégories, les noms des catégories sont plus précis et il met nettement moins de documents dans la catégorie « Other Topics » que STC. Pour approfondir l'analyse, il faudrait connaître et étudier dans le détail les techniques théoriques sous-jacentes de classification de ces algorithmes.

e) Intégration de Carrot² dans Solr

Les versions de Solr à partir de 3.1 intègrent Carrot². Nous avons donc voulu essayer cette intégration.

Pour l'activer, il faut démarrer le serveur Jetty de Solr avec la commande suivante :

```
java -Dsolr.clustering.enabled=true -jar start.jar
```

Les différents algorithmes sont décrits dans le fichier solrconfig.xml

```
<searchComponent name="clustering"
  enable="${solr.clustering.enabled:false}"
  class="solr.clustering.ClusteringComponent" >
```

```

<lst name="engine">
  <str name="name">lingo</str>
  <str name="carrot.algorithm">org.carrot2.clustering.lingo.LingoClusteringAlgorithm</str>
  <str name="carrot.resourcesDir">clustering/carrot2</str>
</lst>
<lst name="engine">
  <str name="name">stc</str>
  <str name="carrot.algorithm">org.carrot2.clustering.stc.STCCLusteringAlgorithm</str>
</lst>
<lst name="engine">
  <str name="name">kmeans</str>
  <str name="carrot.algorithm">org.carrot2.clustering.kmeans.BisectingKMeansClusteringAlgorithm</str>
</lst>
</searchComponent>

```

Pour retrouver les clusters dans Solr, il faut aller à la page Query.

Saisir /clustering dans le champ Request-Handler (qt)

Saisir la requête text:malade dans le champ q

Il est possible de choisir l'algorithme dans le champ Raw Query Parameters en choisissant parmi ceux figurant dans le fichier solrconfig.xml, par exemple clustering.engine=stc.

The screenshot shows the Solr Query interface. The 'Request-Handler (qt)' field is set to '/clustering'. Below it, the 'common' section contains the 'q' field set to 'text:malade'. The 'fq' field is empty. The 'sort' field is empty. The 'start, rows' section shows 'start' set to 0 and 'rows' set to 10. The 'fl' and 'df' fields are empty. At the bottom, the 'Raw Query Parameters' field is set to 'clustering.engine=stc'.

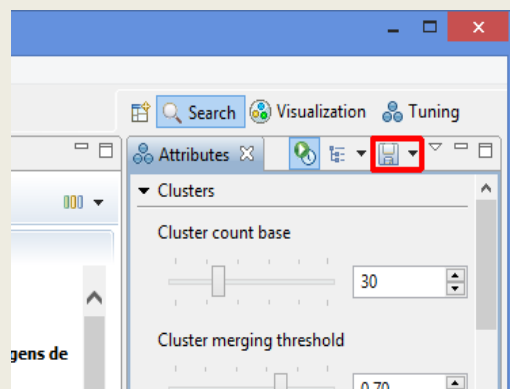

```

    },
    "_version_": 1493656538770833400,
    "score": 0.03698472
  }
},
"clusters": [
  {
    "labels": [
      "L'Illustration"
    ],
    "score": 18.519857589053878,
    "docs": [
      "25310-8",
      "28249-8",
      "43712-8",
      "21199-8",
      "43772-8",
      "44285-8",
      "17248-8",
      "42141-8",
      "42765-8",
      "13622-8",
      "35404-8",
      "15589-8"
    ]
  },
  {
    "labels": [
      "Voyage"
    ],
    "score": 18.147119225551883,
    "docs": [
      "33069-8",
      "46721-8",
      "33675-8",
      "38210-8",
      "38320-8",
      "35309-8",
      "44861-8",
      "44285-8",
      "36331-8",
      "30520-8"
    ]
  },
  {
    "labels": [

```

On retrouve les mêmes clusters qu'avec le workbench.

Il est possible de copier les paramètres du workbench en les sauvegardant dans un fichier xml.



Puis de les charger dans Solr, en mettant le fichier dans conf/clustering/carrot2/ sous le nom lingo-attributes.xml.

5) Conclusion :

Ce projet NFE204 a été pour nous aussi passionnant que le cours. En effet, le cours nous a permis de découvrir de très nombreux domaines qui caractérisent le Big Data (les bases nosql, le mécanisme de map/reduce, la réplication, la distribution, ...) et en particulier celui de la recherche documentaire.

Avant ce cours et ce projet, on était loin d'imaginer la richesse et la complexité de ce domaine, la recherche documentaire.

Certes, nous avons bien conscience d'avoir juste effleuré l'étendue du sujet à travers l'utilisation de Solr et de Carrot². Dès le départ, le domaine de la recherche documentaire nous intéresse vivement en particulier les techniques de similarité. C'est pourquoi, nous avons choisi ce sujet. Mais nous avons été encore plus agréablement surpris par Carrot² et ses résultats de classification.

Nous avons passé beaucoup du temps à faire fonctionner toute la chaîne (récupération de 3 000 œuvres littéraires, développement d'un programme java de transformation en xml, intégration de toute la collection dans la base Solr, couplage avec Carrot² et enfin élaboration de plusieurs tests pour saisir l'apport de Carrot²). Nous sommes vraiment contents d'être allés jusqu'au bout de la mise en pratique d'une partie du cours.

Maintenant que nous avons pris goût aux algorithmes de classification de Carrot², cela nous donne une grande envie d'approfondir ce sujet en particulier de comprendre les techniques sous-jacentes de Lingo, de STC et de Lingo3G de « clustering hiérarchique ». Peut-être que le cours RCP216 que nous venons de commencer apportera dans ce domaine un lot de réponses aussi riches que NFE204.